AUTO AND ASSIGNMENT

Letting the compiler deduce a type

AUTO

Deduced from the initializer

```
auto expenseCount{7};                 // int
auto transactionId{1'000'000};        // int
auto amountSpent{93.33f};             // float, thanks to the f suffix
auto categoryCode{'F'};               // char
auto lifetimeCentsProcessed{9'000'000'000ll}; // long long, thanks to ll
```

`auto` asks the compiler to work out a variable's type from its initializer, instead of us spelling the type out ourselves. Literal suffixes (`u`, `l`, `f`, `ll`, ...) still steer which type gets deduced.

Two different moments

```
transactionId = 1'250'000; // assignment: transactionId already existed
```

The `{...}` braces at declaration are *initializing* a variable - giving it its very first value, the moment it's created. A separate `=` later, without braces, is an *assignment* - replacing a value the variable already has.

WHY BRACES MATTER

Narrowing conversions

```
// int narrowed{93.33}; // compiler error: double doesn't fit losslessly in int

int narrowedAmount = 93.33; // compiles, but silently truncates to 93
```

Braced initialization refuses to silently throw away information - a narrowing conversion won't compile. Plain assignment offers no such protection, which is exactly why this course uses braces `{}` everywhere a variable is declared.

AUTO DOESN'T WATCH YOUR BACK

Deduced once, not re-checked

```
auto receiptsOnFile{250u}; // deduced: unsigned int
receiptsOnFile = -10;      // DANGER: no negative unsigned values, this wraps
```

`auto` deduces a type once, at initialization - it doesn't update if you later assign something that would have deduced differently. Assigning a negative number into an `unsigned` wraps around, the same pitfall from 4.3.

DEBUG IT

Confirm what auto actually picked

Hover `transactionId` or `amountSpent` in your IDE while paused at a breakpoint - most debuggers show the deduced type right in the tooltip, which is the fastest way to double-check `auto` picked what you expected.